

Document 05 — LangChain Basics

This is for self-study and live mentoring. Full plan: [CURRICULUM.md](#)

Prerequisites

[04_openai_api](#) (Project 1 done)

How to Read & Practice This Document

- **What:** building LLM calls using LangChain's tools (LCEL, prompt templates, output parsers).
- **Why:** the code you write later for real agents — including LangGraph — is built on these exact same basic pieces.
- **When:** when you need to reuse and combine many prompts or chains, not for a single one-off call (where the raw SDK is often simpler — you'll need to be able to explain which one to pick and why).
- **How to practice:** 1. Read the material once, just to get the shape of it. 2. Do the **Basic** exercise closed-book. 3. Do the **Intermediate/Real-world/Edge case** exercises with notes open. 4. Build the chain without copying Doc04's raw-SDK version — rebuild it yourself, don't paste. 5. Use **Hint 1** → **Hint 4** only after really trying. Ask for the full solution only if you say "**Show me the solution.**" 6. Before moving on, explain out loud when you'd pick LangChain over the raw SDK for a new task, and why. If you can't, you're not done.

Jump to: [Core Concepts](#) · [Practice Exercises](#) · [Build Task](#)

The Story — what this document is actually building

Picture this: in Doc04 you built a chatbot by hand — building a `messages` list yourself, calling the OpenAI API yourself, reading the reply yourself, every single time. That worked, but every new feature meant rewriting the same plumbing again from scratch.

This document is about a library, LangChain, that takes exactly that repeated plumbing and turns it into small, swappable pieces you connect instead of rewrite. **First**, `ChatOpenAI` replaces your raw API call — same model, same underlying request, just a standard shape that works the same way no matter which AI provider you're actually using underneath. **Second**, a prompt template separates the fixed wording of a prompt from the parts that change each time, so the prompt itself becomes something you can log, test, and swap out — not a string you build by hand with f-strings scattered

through your code. **Third**, LCEL's `|` symbol lets you snap a prompt template, a model, and an output parser together into one pipeline — `prompt | model | parser` — so calling it once runs all three steps in order, and the whole thing behaves like one reusable object. **Fourth**, the output parser is what turns the model's raw text reply into a real Python object your code can actually use, and it checks that shape *after* the model answers — a different method from Doc04's structured output, which shapes the answer *while* it's being generated.

That's the whole story: each Core Concept below is one interchangeable piece, and LCEL is the glue that snaps them together. The Build Task at the end asks you to build exactly that — one small, reusable chain module — and prove, side by side, that it gives the same answer as Doc04's raw version, so you can actually explain, not just guess, when the extra layer is worth its cost.

Core Concepts (read this first — everything you need is here)

Topics on this page: [What a library gives you, and what it costs](#) · [ChatOpenAI](#) · [Prompt templates](#) · [LCEL](#) · [Output parsers](#) · [The three LangChain packages](#)

What a library gives you, and what it costs

Doc04 taught the raw way: build a `messages` list by hand, call the API, read the reply by hand, every single time. LangChain's value is that it standardizes and reuses all that repeated work — building prompts, reading replies, swapping which AI provider you use — behind shared, common tools, so you're not rewriting the same boring code for every new task. **The cost:** there's now an extra layer between your code and the actual API call, so when something breaks, you're debugging through both LangChain's code *and* the API, not just the API. **Why this trade-off is worth naming clearly** (and why you'll be asked to explain it, not just repeat it): for a single one-off call, the raw SDK is often genuinely simpler and easier to debug. LangChain earns its cost when you have many similar prompts or chains that benefit from a shared, swappable structure — not for a quick one-time script.

ChatOpenAI : a wrapper, not a different model

`ChatOpenAI` is LangChain's version of the same Chat Completions API from Doc04 — same model, same underlying HTTP call, just a different way of calling it in Python (`.invoke(messages)` instead of `client.chat.completions.create(...)`). **Why it's built this way:** LangChain supports many different AI providers behind one shared way of calling them, so code written for `ChatOpenAI` looks a lot like code written for another provider's wrapper — switching providers should mostly mean changing one line, not rewriting your whole project.

Prompt templates: keeping the fixed part separate from the changing part

A prompt template separates the *fixed* wording of a prompt from the *changing* parts you'll fill in later — `ChatPromptTemplate.from_template("Extract the {field} from: {text}")` instead of

building the string by hand with f-strings scattered everywhere. **Why this matters beyond tidiness:** it makes prompts into real, checkable objects — you can log the exact prompt that was sent, save different versions, and test them — instead of string-building logic hidden inside random functions. **How it fails, and where:** a template that refers to a variable you never provided raises an error *right when it builds the prompt*, before any API call even happens — which is a cheaper, faster failure than sending a broken prompt to the model and getting a confusing answer back.

LCEL: chaining pieces together with `|`

LCEL (LangChain's chaining syntax) lets you connect a prompt template, a model, and an output parser into one pipeline using the `|` symbol: `chain = prompt | model | parser`. Calling `chain.invoke(inputs)` runs all three steps in order, passing each one's output into the next. **Why this exists, instead of just calling each piece by hand:** the whole chain becomes one single, reusable object (called a `Runnable`) that works the same way no matter how many steps you've connected — `.invoke()`, `.stream()`, `.batch()` all just work — and it can be plugged *into* bigger chains the same way. **How it works underneath:** every piece follows the same shared `Runnable` pattern. `|` is really just shorthand for "send this piece's output into the next piece's input" — there's no hidden magic beyond that, and it's worth remembering that so LCEL never feels like a black box.

Output parsers: turning raw text into a real Python object

An output parser takes the model's raw reply and turns it into a structured Python type your code can actually use — a plain text parser just pulls out the text; a Pydantic/structured parser checks the reply against a shape, and raises a clear, named error if it doesn't match. **Why this is a different idea from Doc04's structured outputs:** LangChain's parser checks the result *after* it's generated (catching a mismatch no matter how the text was produced), while OpenAI's built-in structured output limits the *writing itself* — two different methods, and you'll compare them directly in this document's build task.

The three LangChain packages: `langchain-core`, `langchain-openai`, `langchain`

LangChain is split into separate installable packages by purpose: `langchain-core` holds the basic building blocks (the `Runnable` pattern, prompt templates, output parsers) with very few extra dependencies. `langchain-openai` holds the OpenAI-specific pieces (`ChatOpenAI` and friends). The main `langchain` package holds bigger, higher-level chains, agents, and helpers built on top of both. **Why this split exists:** so a project that only needs the basic building blocks plus one provider doesn't have to install every provider's library and every older tool — it's a way of keeping dependencies clean, and it matters once you're reading real production code and need to understand why a project imports from three different `langchain_*` packages instead of one.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- [LangChain — Introduction](#) — what LangChain is, and isn't.
- [LangChain — Concepts hub](#) — start with chat models, prompt templates, LCEL, output parsers.
- [LangChain Academy](#) — a free official course; do the intro alongside this document.

Practice Exercises

Setup for this document's practice code: work inside `05_langchain_fundamentals/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install langchain langchain-openai langchain-core`.

How to run each exercise: group your practice code by topic, not by difficulty level. If two exercises below are really about the same thing, save them together in ONE script named after that topic — for example, if two exercises are both about `.env` config, save both in one file like `env_config_practice.py`, with each level's version as its own clearly labeled section inside it. Run each topic's file directly, for example: `python structured_output_practice.py`.

For this document, save your practice code as: - **Basic** (your first LCEL chain) is its own topic — save it as `lcel_chain_basics_practice.py`. - **Intermediate** (swap in a structured parser) and **Failure** (force the parser to actually fail) are both about structured-output parsing and its errors — save them together as `structured_output_practice.py`, one section per level. - **Real-world** (rebuild a Project 1 feature, LCEL-style) is its own topic — save it as `lcel_vs_raw_sdk_practice.py`. - **Edge cases** (a template with the wrong variables) is its own topic — save it as `prompt_template_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to advanced, side by side.

Jump to an exercise: [Basic](#) · [Intermediate](#) · [Real-world](#) · [Edge cases](#) · [Failure](#) · [Build Task](#)

Basic — your first LCEL chain

- **What:** connect a prompt template → `ChatOpenAI` → text output parser with `|`.
- **Why:** this 3-piece chain is the atom every LangChain thing you'll ever build is made of — get comfortable with it before adding anything else.
- **When you'll hit this for real:** this document's own Build Task, and every chain you write from here forward.

- **How to code it:** `prompt = ChatPromptTemplate.from_template("Answer: {question}")`, `chain = prompt | ChatOpenAI() | StrOutputParser()`, then `chain.invoke({"question": "..."})`.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Intermediate — swap in a structured parser

- **What:** replace the text parser with a Pydantic/structured one, and see what happens when the model's answer doesn't match the shape.
- **Why:** this is where you feel, directly, the difference between LCEL's after-the-fact parser checking and Doc04's during-generation structured output.
- **When you'll hit this for real:** any time you're extracting data instead of just displaying text — which is most of what agents actually do.
- **How to code it:** define a small Pydantic model, use `.with_structured_output(YourModel)` on the chat model, run it on a clearly-matching input, then a deliberately mismatched one — read the exact error.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Real-world — rebuild a Project 1 feature, LCEL-style

- **What:** take one feature from Project 1 (the raw-SDK version) and rebuild it with LCEL, keeping both versions.
- **Why:** this is the only way to actually compare LangChain vs. raw SDK — reading about the trade-off isn't the same as having both versions of the same feature side by side.
- **When you'll hit this for real:** this document's own Build Task asks for exactly this comparison.
- **How to code it:** copy Project 1's structured-extraction feature into a new file, rewrite it as `prompt → model → parser` with `|`, then write a script that runs both on the same 3 inputs and diffs the outputs.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Edge cases — a template with the wrong variables

- **What:** a prompt template missing a variable it needs, and one given an extra variable it doesn't use — see what fails, and exactly when.
- **Why:** knowing whether a broken template fails *while building the prompt* or *only once the API call goes out* changes how fast you can debug it.
- **When you'll hit this for real:** a refactor where you rename a variable in the template but forget to update where it's called from — a real, easy-to-make mistake.
- **How to code it:** call `.invoke({})` on a template expecting `{question}` — read the exact error. Then call it with `{"question": "...", "extra": "..."}` and confirm whether it's silently ignored or errors.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Failure — force the parser to actually fail

- **What:** ask the model something that won't fit your Pydantic shape on purpose, and read the real error the parser raises.
- **Why:** you need to know, before it happens in production, exactly what kind of exception your code needs to catch here.
- **When you'll hit this for real:** any structured-extraction feature, on an input ambiguous enough that the model can't cleanly produce your expected shape.
- **How to code it:** ask a structured-output chain expecting a number field for something that's clearly a description, not a number — catch the resulting error and print its type.
- **Stuck?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Build Task — Reusable Chain Module

Stuck on the Build Task? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

Goal: a small chain module that later documents will reuse instead of writing raw SDK calls each time — a comparison point next to Project 1, not a replacement for it.

Requirements:

- One LCEL chain that wraps a prompt template, a model, and a parser as one callable piece.
- The model choice (name, temperature) comes from your config — never hardcoded.
- A side-by-side test/script that proves your LCEL version and Doc04's raw-SDK version give the same answer for the same input.

Inputs: the same structured-extraction task as Project 1's structured-output feature.

Outputs: the same typed object as Project 1's, but built a different way.

Constraints: don't delete or replace the Doc04 raw version — you need both, to compare them and to answer the interview question "when would you use which."

Suggested files:

```
05_langchain_fundamentals/
├─ chain.py
├─ prompts.py
├─ compare_with_raw_sdk.py
```

Functions/Components to build:

- `prompts.py` → the `PromptTemplate` / `ChatPromptTemplate`
- `chain.py` → `build_extraction_chain()` → `Runnable`

- `compare_with_raw_sdk.py` → runs both versions on the same inputs, prints the difference

Expected Behavior

- The chain gives the same structured object as Project 1's raw version, for the same input.
- Changing the prompt template's variables shouldn't require touching the chain-building code.

Test Cases

Scenario	Expected
A well-formed input	Chain and raw-SDK results match
A missing template variable	A clear error while building the prompt, not silently left blank
The model's answer doesn't fit the shape	The parser raises a clear, named, catchable error

Break-It / Debug Preview

- A chain that quietly returns the wrong Python type, because the parser doesn't actually match what the prompt asks for.
- Full debugging drill in [14_debugging_lab](#).

Interview Topics Preview

- When LangChain is worth its cost vs. when the raw SDK is simpler and easier to debug · what LCEL actually connects when it runs · `langchain-core` vs. `langchain-openai` vs. `langchain`.

Move On When

You can decide, for a given task, LangChain vs. raw SDK — not just default to whichever you learned last. Full details: [CURRICULUM.md §4](#).

Stuck? Ask for **Hint 1** or **Hint 2** (each has Basic/Intermediate/Advanced depth). Ask for the full solution only if you say **"Show me the solution."**